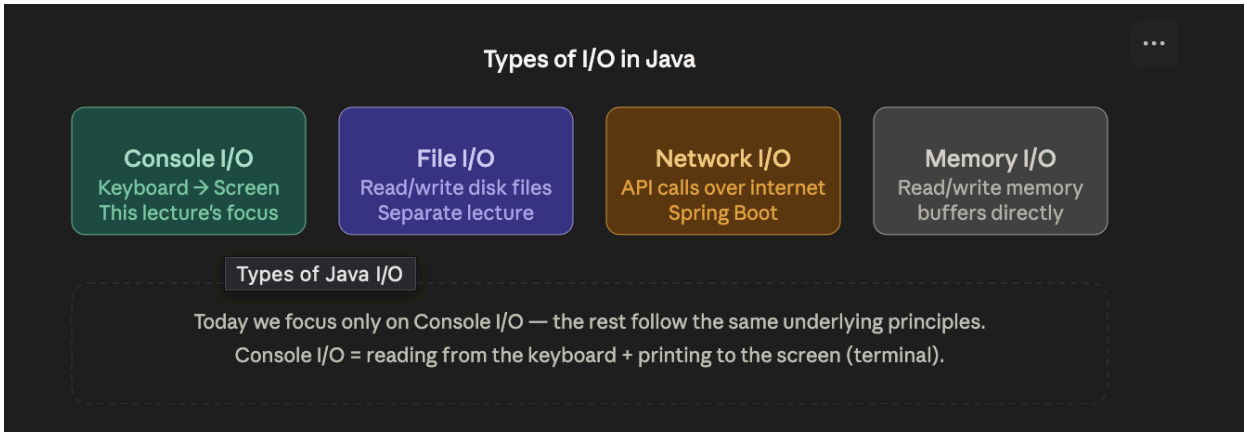
# Part 1: What is Standard Input/Output?

Every program needs to **communicate with the outside world**. Before this lecture, your Java programs had all their data hardcoded inside them. But real programs need to:

- **Receive** data from somewhere (keyboard, file, network, memory)
- **Send** data to somewhere (screen, file, network, memory)

This is called **I/O** — Input/Output.

Java categorizes I/O by *where* the data comes from or goes to:



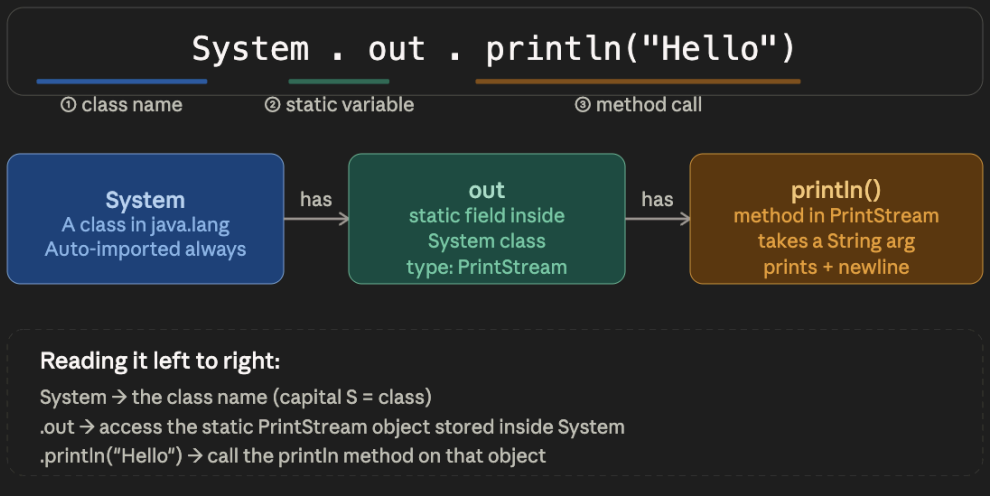
## Part 2: Understanding `System.out.println()` from First Principles

You've been writing this line since day one:

```java
System.out.println("Hello");
```

But what is *actually happening*? Let's dissect it completely.

The key insight is: **this is not magic syntax — it's just a method call on an object.** Let's build it up piece by piece.




**Reading it left to right:**

System → the class name (capital S = class)
.out → access the static PrintStream object stored inside System
.println("Hello") → call the println method on that object

Now let's understand **why** `out` is declared `static`. This is a crucial OOP point:

```java
// What actually exists inside the System class (simplified):
public class System {
    public static final PrintStream out = ...;  // static!
    public static final PrintStream err = ...;
    public static final InputStream in  = ...;
}
```

Because `out` is `static`, you access it through the **class name** directly — `System.out` — without needing to create a `System` object first. If it were NOT static, you'd have to do:

```java
System s = new System();   // create object first
s.out.println("Hello"
```

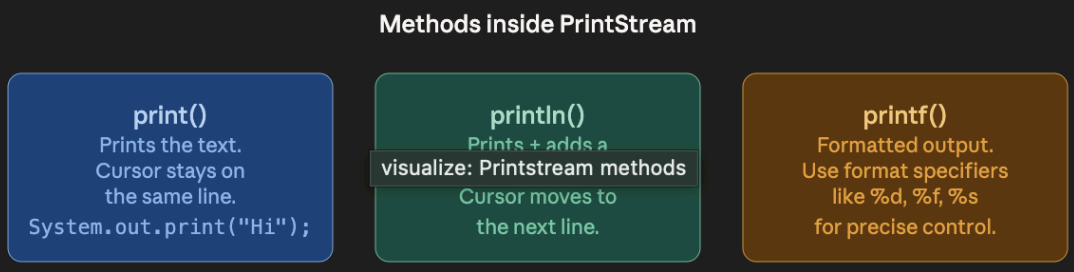
Anatomy of System.out.println

rough object

Java made it static so you can just write `System.out.println()` directly. Much cleaner.

## Part 3: The `PrintStream` Class — What `out` Actually Is

`out` is not just any object — it's a `PrintStream` object. `PrintStream` lives in the `java.io` package and gives you multiple ways to write output:




```
System.out.print("A"); System.out.print("B"); // output: AB
System.out.println("A"); System.out.println("B"); // output: A
// B
System.out.printf("%.2f", 3.14159); // output: 3.14
```

## Part 4: `System.err` — The Error Stream

Inside the `System` class, there's a second `PrintStream` called `err`. It's the same type as `out` — but it's meant for **error messages**.

```java
System.out.println("User logged in");       // normal output
System.err.println("Invalid age entered");  // error output
```

**Why does this separation matter?**

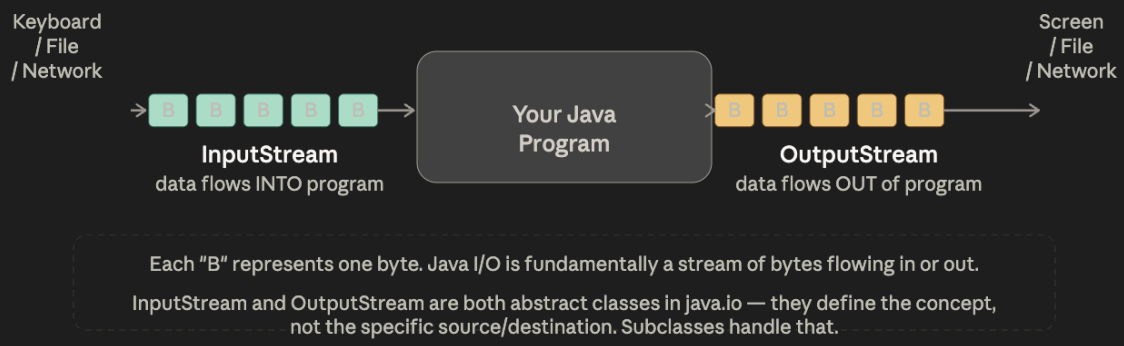
- It lets you distinguish errors from normal logs just by looking at the code
- The operating system treats them as *different streams* — error output can be shown in red, logged s PrintStream methods comparison different file
- In real applications, error logs go to `error.log` while normal output goes to `app.log`

In practice, when you're learning, you'll use `System.out` for everything. But in production code, use `System.err` for genuinely exceptional/error situations.

## Part 5: What is a Stream?

Before we can understand input, we need to understand **streams** — because Java's entire I/O system is built on this concept.

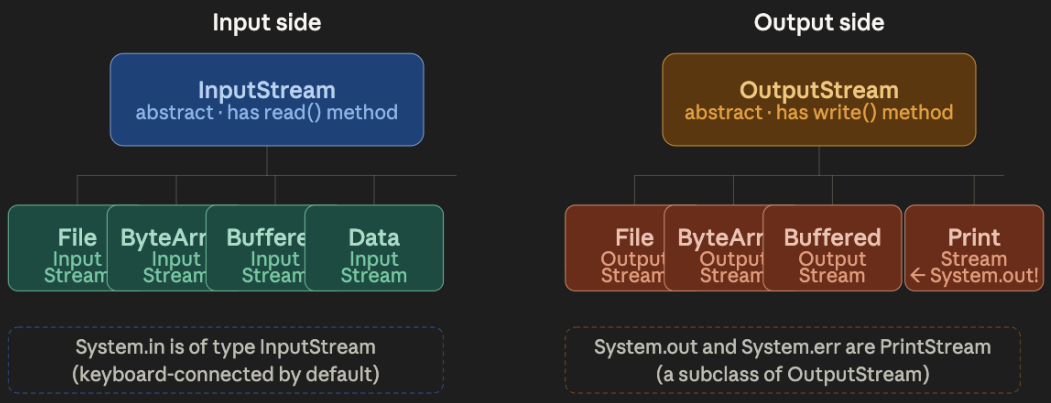
A **stream** is simply a *flow of data*. Imagine a water pipe: data flows through it in one direction, one piece at a time.




The critical insight: **all Java I/O is a stream of bytes**. Even when you type the letter 'A' on your keyboard, Java receives the number 65 (its ASCII value) as a byte. We'll come back to this.

# Part 6: The Full InputStream / OutputStream Hierarchy

`InputStream` and `OutputStream` are **abstract classes** — they define *what* can be done (read/write), but not *how* or *where*. Their subclasses handle the specifics.




System.in is of type InputStream (keyboard-connected by default)

System.out and System.err are PrintStream (a subclass of OutputStream)

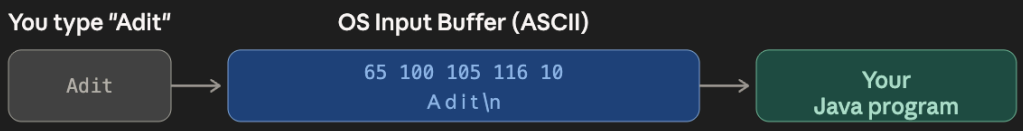
**Why abstract parent classes?**

InputStream just says: "you can read()". It doesn't say where from.
FileInputStream overrides read() to read from a file.
BufferedInputStream overrides it to read from a buffer.
Each subclass handles a different source/destination — same contract, different implementation.

## Part 7: `System.in.read()` — The Primitive Way to Read Input

Now let's take input from the user. `System.in` is an `InputStream` object, connected to your keyboard by default. Its method `read()` reads **one byte at a time**.

Here's the critical thing to understand about how the keyboard works:




**The problem: read() reads ONE byte per call**

```
int x = System.in.read(); // reads 65 → the letter 'A'
```

To read "Adit" you'd need FOUR separate read() calls — one per character

**What you'd have to write (very tedious):**

```
String s = "";
int c;
while ((c = System.in.read()) != '\n') {
s += (char) c; // cast int→char each time
} // need a loop just to read one word — messy!
```

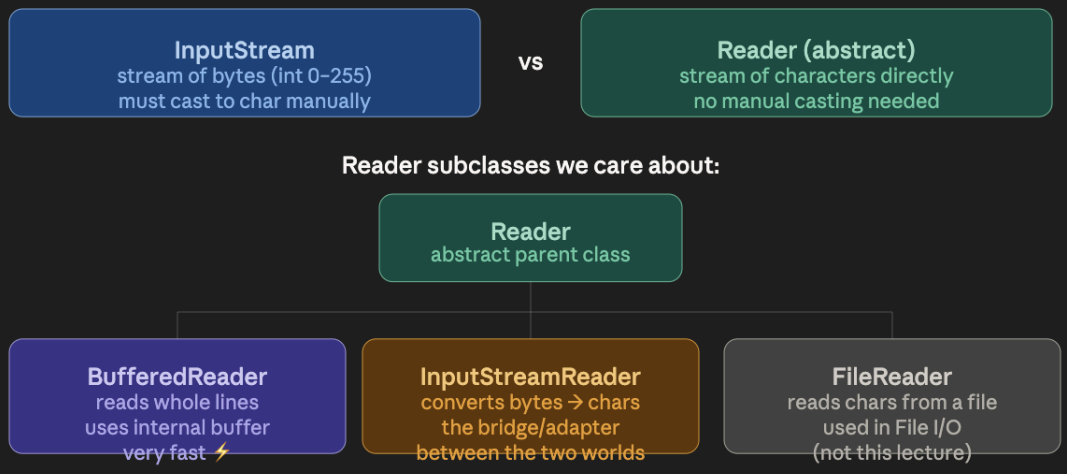
Also important: `read()` returns an `int`, not a `char`. Because Java's `char` is 2 bytes (Unicode), but keyboard input is 1 byte (ASCII). You have to manually cast `(char)` to convert. This is why `System.in.read()` alone is not practical for real programs.

## Part 8: The `Reader` Class — Moving to Characters

Java recognized that reading **bytes** and manually converting to **characters** was painful. So a whole new class hierarchy was introduced: the `Reader` family.

The key difference:

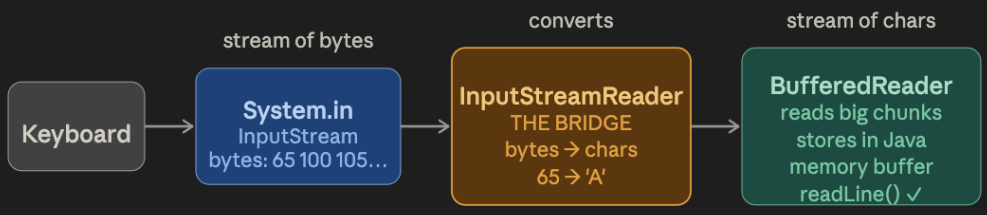
- `InputStream` → stream of **bytes** (raw binary)
- `Reader` → stream of **characters** (already decoded — no manual casting needed)



## Part 9: `InputStreamReader` — The Bridge Between Bytes and Characters

Here's the core problem: `System.in` gives us a **byte stream** ( `InputStream` ), but `BufferedReader` wants a **character stream** ( `Reader` ). They're incompatible directly.

`InputStreamReader` solves this. It wraps an `InputStream` and converts its bytes to characters — it's the **adapter/bridge** between the two worlds.

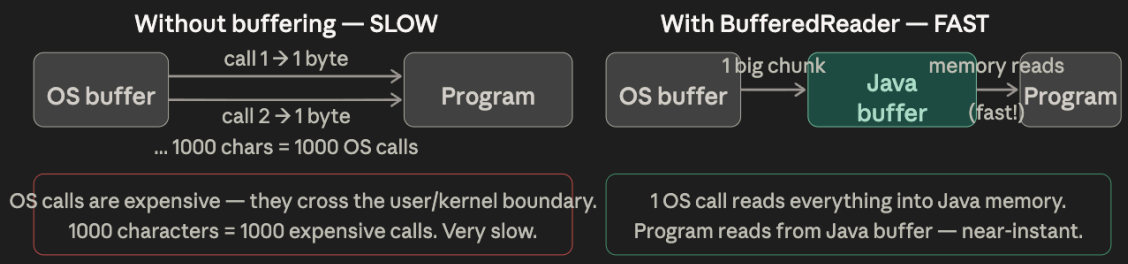


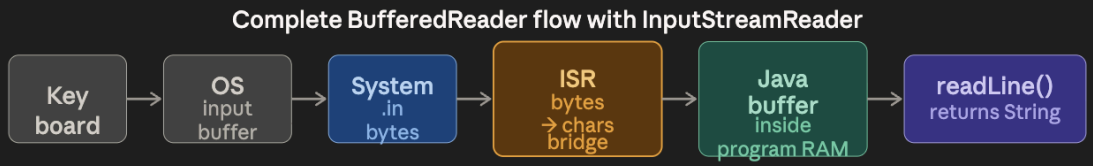

**The code (all three layers):**

```
InputStreamReader isr = new InputStreamReader(System.in);
BufferedReader br = new BufferedReader(isr);
```

## Part 10: `BufferedReader` — How Buffering Works Internally

This is where performance becomes important. Let's understand **why** `BufferedReader` is so much faster than calling `System.in.read()` repeatedly.




**Full code to read a line with BufferedReader:**

```
InputStreamReader isr = new InputStreamReader(System.in);
BufferedReader br = new BufferedReader(isr);
String name = br.readLine(); // reads the whole line at once!
// or combine into one line:
```

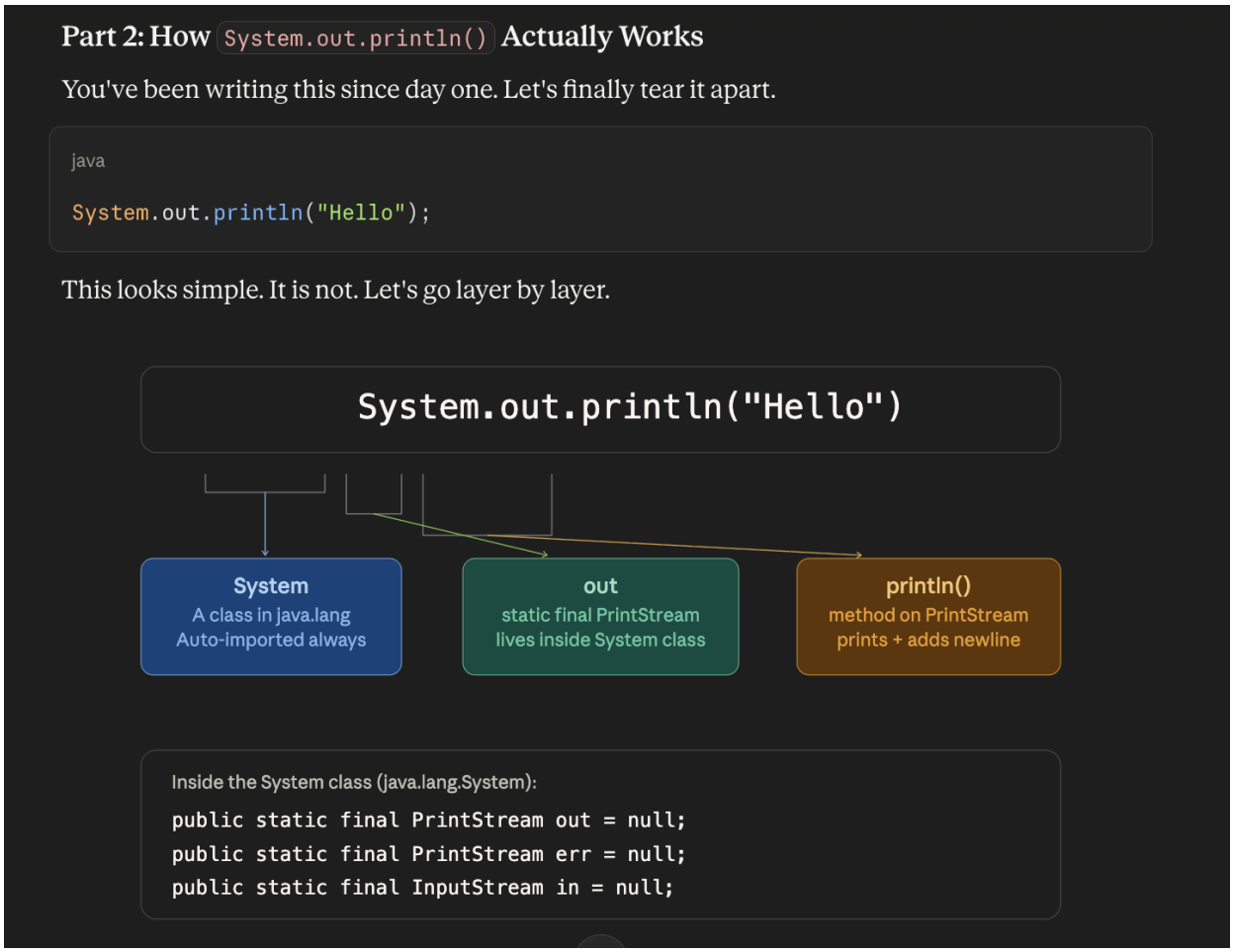
## Part 1: What is Standard Input/Output?

When you write a Java program, it doesn't exist in isolation. It lives inside a computer that has a keyboard, a screen, files, a network. "Standard I/O" means the default channels through which a program communicates with the outside world.

Before understanding *how* Java does I/O, you need to understand *what kinds* of I/O exist:

**Console I/O** — keyboard input, screen output. This is what we focus on today. **File I/O** — reading from or writing to files on disk. **Network I/O** — sending/receiving data over the internet (API calls, HTTP requests). **Memory I/O** — reading from in-memory buffers.

All of these use the same underlying concept: **streams**.




**Breaking it down word by word:**

`System` — this is a **class**. It starts with a capital letter, so you already know. It lives in the `java.lang` package, which Java automatically imports for every file you write. You never have to `import java.lang.System` — it's always there.

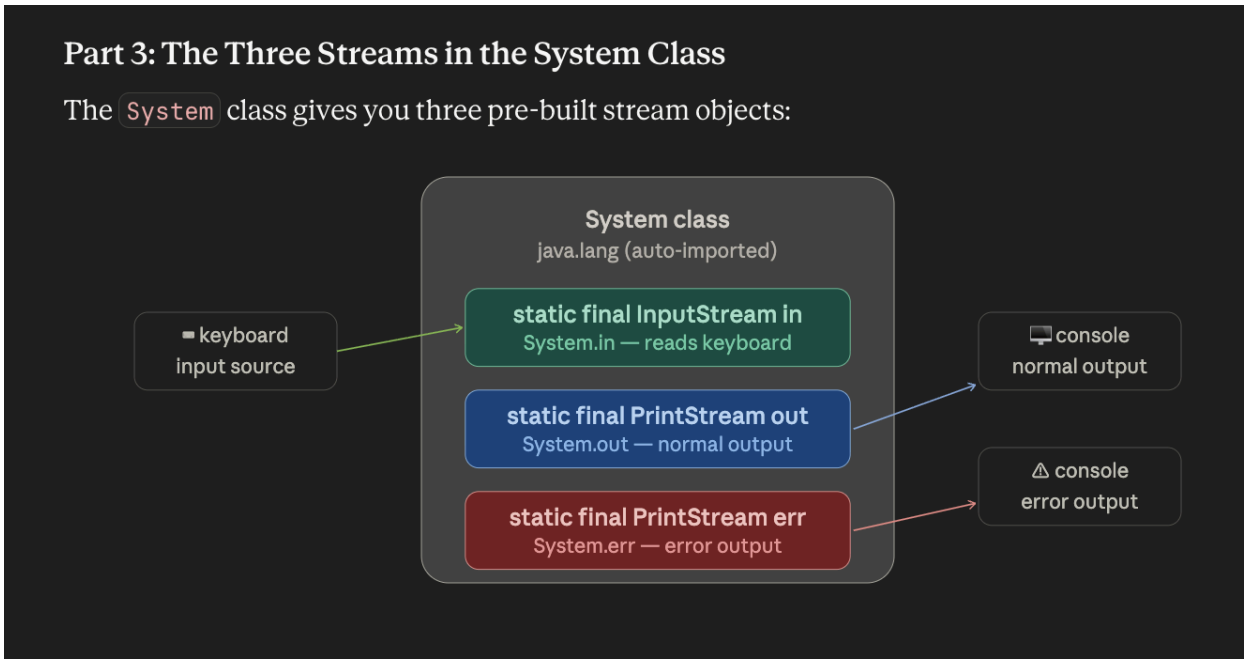
`out` — this is a **static reference variable** declared inside the `System` class. Since it's static, you access it via the class name directly: `System.out`. You don't need to create an object of `System` first. Its type is `PrintStream` — a class Java provides for printing to output streams.

`println("Hello")` — this is a **method** defined inside `PrintStream`. It takes your string, writes it to the console, and appends a newline character `\n` at the end so the next print starts on a fresh line.

So the full read is: *"On the `out` object (of type `PrintStream`) that lives statically inside the `System` class, call the `println` method with the argument `"Hello"`."*

**Why is `out` declared as `static final`?**

- `static` — so you can access it via `System.out` without creating a `System` object first
- `final` — so nobody can replace it with a different stream accidentally
- `public` — so any code in any class can use it




**`System.out`** — for all normal program output. Use this for displaying results, messages, data.

**`System.err`** — also a `PrintStream`, so it has the exact same methods (`println`, `print`, `printf`). But it's *semantically* for errors and exceptional conditions. Why does this matter?

- It signals intent to other developers reading your code
- Tools like log viewers and terminals often show `err` output in **red**, separately from normal output
- The operating system treats them as separate streams — you can redirect them independently

In practice: when something goes wrong and you want to signal it's an error, use `System.err.println("Invalid age")` instead of `System.out.println`.

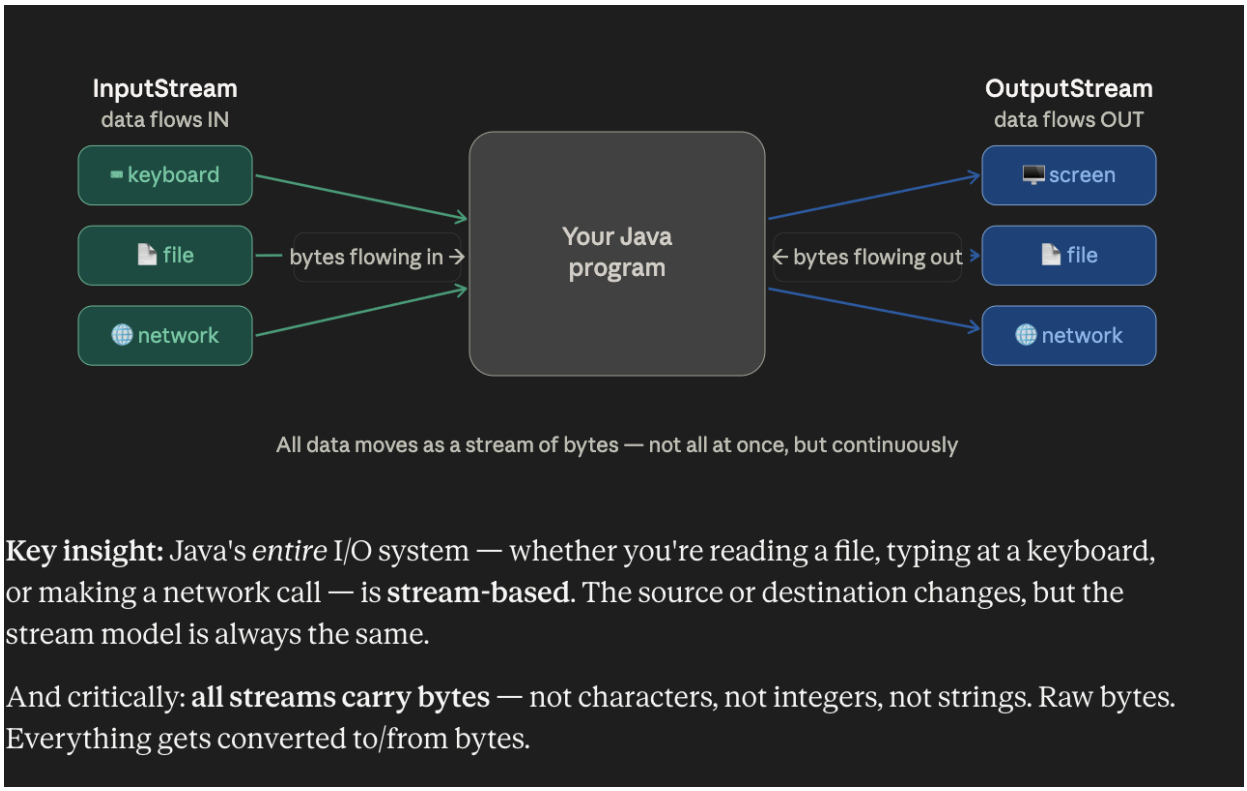
**`System.in`** — for reading keyboard input. Its type is `InputStream` — we'll dig into this deeply in a moment.

## Part 4: What Are Streams?

This is the most important concept to understand before anything else. Everything in Java I/O is built on streams.

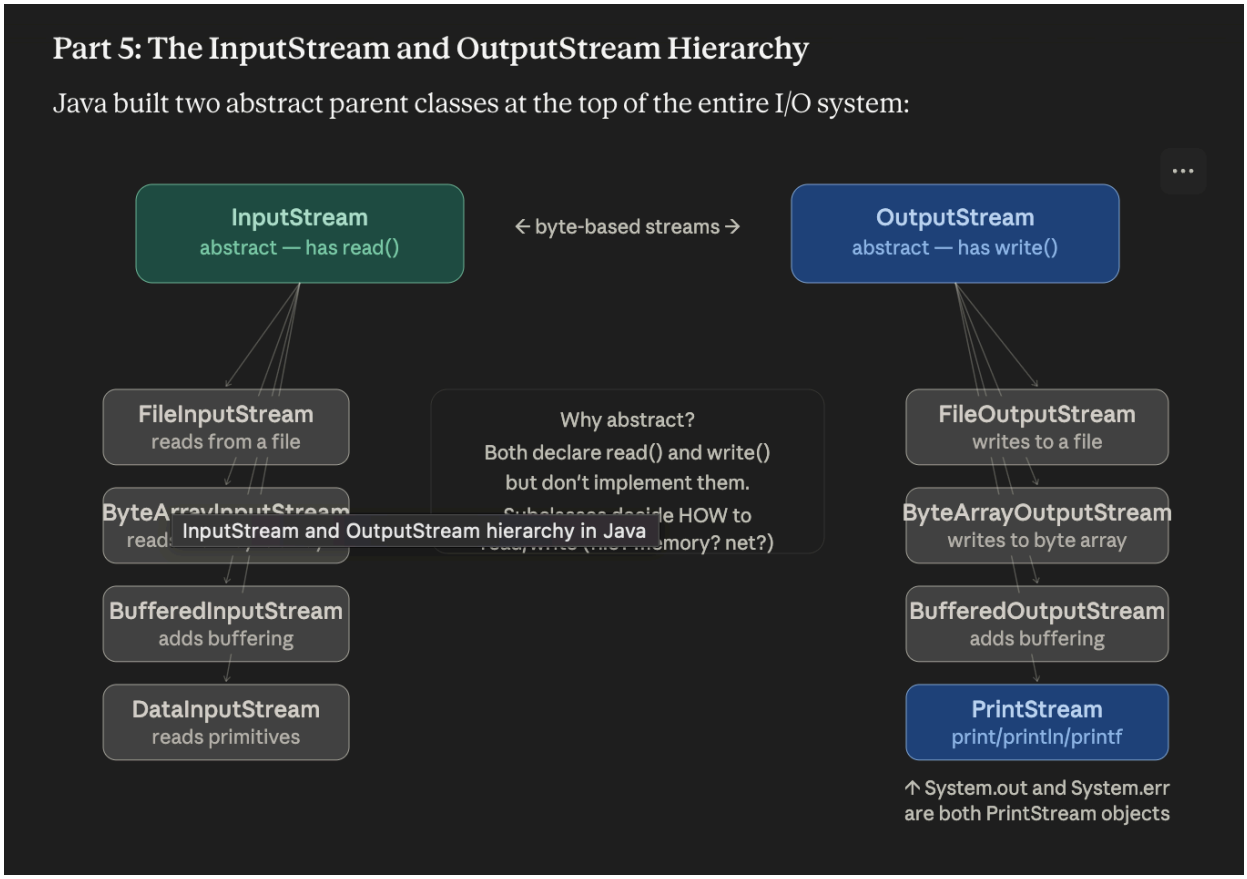
**A stream is a flow of data.** Think of a water pipe. Data flows through it — one end produces data (the source), the other consumes it (the destination). The data doesn't jump all at once; it flows continuously.

There are exactly two directions:




**Key insight:** Java's *entire* I/O system — whether you're reading a file, typing at a keyboard, or making a network call — is **stream-based**. The source or destination changes, but the stream model is always the same.

And critically: **all streams carry bytes** — not characters, not integers, not strings. Raw bytes. Everything gets converted to/from bytes.




**Why are they abstract?**

`InputStream` says: "I promise you'll be able to `read()` data." But *where* that data comes from — keyboard, file, network — it doesn't know. That's the subclass's job.

`OutputStream` says: "I promise you'll be able to `write()` data." But *where* it goes is up to the subclass.

This is the **abstraction** from OOP in action. The parent defines the contract. The children implement the detail.

## Part 6: How `System.in.read()` Works — Reading Raw Bytes

Now we actually try to take input. The most primitive way:

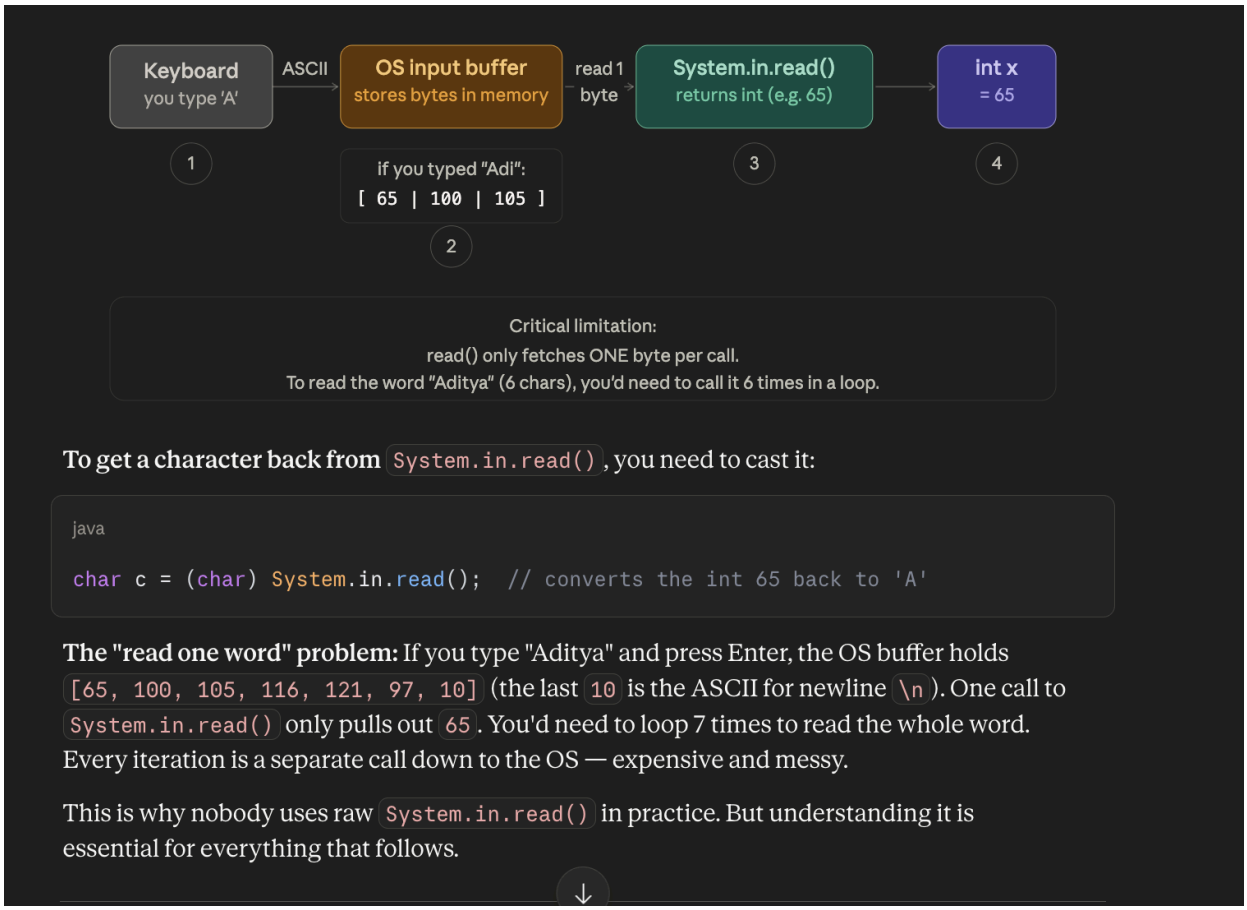
```
int x = System.in.read();
```

`System.in` is an `InputStream` object, connected by default to your keyboard. The `read()` method reads **exactly one byte** and returns it as an `int` (not a `char` — important!).

**Why does it return `int` instead of `char`?**

When you press 'A' on your keyboard, the keyboard sends Java the ASCII value of A, which is 65. That 65 arrives as a byte. Java stores it as an `int` because `read()` also uses `-1` to signal "there's nothing left to read" (end of stream), and a `byte` can't store `-1` distinctly from valid data. So `int` gives enough room.

**The data flow looks like this:**




## Part 7: The Reader Class — Switching from Bytes to Characters

Java realized the byte-by-byte approach was clunky. So it introduced a parallel hierarchy: the **Reader classes**, designed to work with *characters* instead of raw bytes.

v

visualize

Let me continue with the explanation in text and rebuild the key diagrams.

---

The `Reader` class hierarchy is separate from `InputStream`. Where `InputStream` deals in **bytes** (raw numbers 0-255), `Reader` deals in **characters** (Java `char`, which is Unicode, 2 bytes wide). This distinction matters because keyboards produce characters, not just bytes.

**The three important classes under `Reader`:**

- `Reader` — abstract parent, declares `read()` returning a `char`
- `InputStreamReader` — the bridge between the byte world and the character world
- `BufferedReader` — adds buffering for performance
- `FileReader` — for reading character data from files

---

## Part 8: The Compatibility Problem — Why InputStreamReader Exists

Here's the problem: `System.in` is an `InputStream` (byte stream). `BufferedReader` works with character streams. They're incompatible — you can't plug one directly into the other.

**`InputStreamReader` is the adapter (bridge) that solves this.**

It takes an `InputStream` (byte stream) and converts it into a character stream that `BufferedReader` can consume.

Internally, it decodes bytes using a character encoding (by default, the platform encoding, usually UTF-8). So when `System.in` delivers the byte `65`, `InputStreamReader` decodes it as the character `'A'`.

---

## Part 9: BufferedReader — How It Fixes the Performance Problem

**The problem with `System.in.read()`:** Every call to `read()` goes down to the OS and asks "give me one byte." If the user typed 1000 characters, that's 1000 OS calls. OS calls are expensive — they involve switching from user mode to kernel mode.

**BufferedReader's solution:** It reads a large *chunk* of data from the OS all at once, stores that chunk in memory (a buffer — think of it as a temporary array), and then serves your program from that in-memory buffer. Your program never needs to call the OS again for those bytes.

v

visualize

Let me keep going with text explanations and try smaller diagrams.

**The data flow with BufferedReader:**

1. You type "Aditya" and press Enter
2. The OS stores bytes in its buffer: `[65, 100, 105, 116, 121, 97, 10]`
3. `System.in` (an `InputStream`) receives those bytes
4. `InputStreamReader` converts: bytes → characters (decoding ASCII/UTF-8)
5. `BufferedReader` reads a whole chunk into its internal memory buffer
6. `br.readLine()` returns the entire string `"Aditya"` at once

**The code:**

java
```
import java.io.*;

public class Demo {
    public static void main(String[] args) throws IOException {
        InputStreamReader isr = new InputStreamReader(System.in);
        BufferedReader br = new BufferedReader(isr);

        String name = br.readLine(); // reads the whole line
        System.out.println(name);
    }
}
```
Or more commonly written in one line (no separate isr variable):
java
```
BufferedReader br = new BufferedReader(new InputStreamReader(System.in));
String name = br.readLine();
```

The `new InputStreamReader(System.in)` object is created on the heap and its reference is passed directly into `BufferedReader`'s constructor. You just don't bother storing it in a named variable.

---

## Part 10: Limitations of BufferedReader

`BufferedReader` is fast, but painful to use:

**Problem 1 — Only reads Strings.** `readLine()` always returns a `String`. If you want the user to enter their age (an integer), you have to do this two-step process:

java
String line = br.readLine();       // reads "25" as a String
int age = Integer.parseInt(line);  // converts "25" → 25

`Integer.parseInt()` is a static method on the `Integer` wrapper class that converts a string representation of a number into an actual `int`. You have to do this every time.

**Problem 2 — Verbose setup.** Just to read one line from the keyboard, you write two lines of constructor code, and you have to wrap `main` with `throws IOException`. This is ceremony that gets in the way.

**Problem 3 — Breaks Java's promise of simplicity.** Other languages let you read input in one line. Java 1.0 through 1.4 made you write a paragraph. This was a real criticism of the language.

---

## Part 11: Scanner — The Modern Solution

Java 1.5 (released 2004) introduced `Scanner`. It lives in the `java.util` package (utilities), *not* `java.io` — because it's not really an I/O class. It's a utility class that wraps the I/O layer and gives you a clean, simple interface.

**Creating a Scanner:**

java
import java.util.Scanner;

Scanner sc = new Scanner(System.in);  // connect to keyboard

`Scanner`'s constructor accepts an `InputStream` — and that's where `System.in` goes. So under the hood, Scanner still uses the same byte stream. It just hides all the complexity from you.

**Reading different types:**

java
String name   = sc.nextLine();    // reads entire line including spaces
String word   = sc.next();        // reads one word (stops at whitespace)
int    age    = sc.nextInt();     // reads an integer directly

```
double price  = sc.nextDouble();  // reads a double
boolean flag  = sc.nextBoolean(); // reads "true" or "false"
long   big    = sc.nextLong();    // reads a long
float  f      = sc.nextFloat();   // reads a float
```

No manual type casting. No `Integer.parseInt()`. Scanner handles it all internally.

---

## Part 12: How Scanner Tokenizes Input

Scanner doesn't just read characters — it **tokenizes** the input. Tokenization means it splits the input into meaningful chunks (tokens) using whitespace as the delimiter by default.

If you type: `hello I am Aditya`

Scanner sees four tokens: `"hello"`, `"I"`, `"am"`, `"Aditya"`.

- `sc.next()` gives you one token at a time
- `sc.nextLine()` gives you everything until the newline (the whole line, not just one token)

**A subtle trap — `nextInt()` followed by `nextLine()`:**

```
java
int age = sc.nextInt();      // reads the number, but leaves '\n' in buffer
String name = sc.nextLine(); // immediately reads the leftover '\n' as an empty line!
```

This is one of the most common bugs beginners hit. `nextInt()` consumes the number but leaves the newline character in the buffer. When you then call `nextLine()`, it sees that leftover `\n` and returns an empty string immediately.

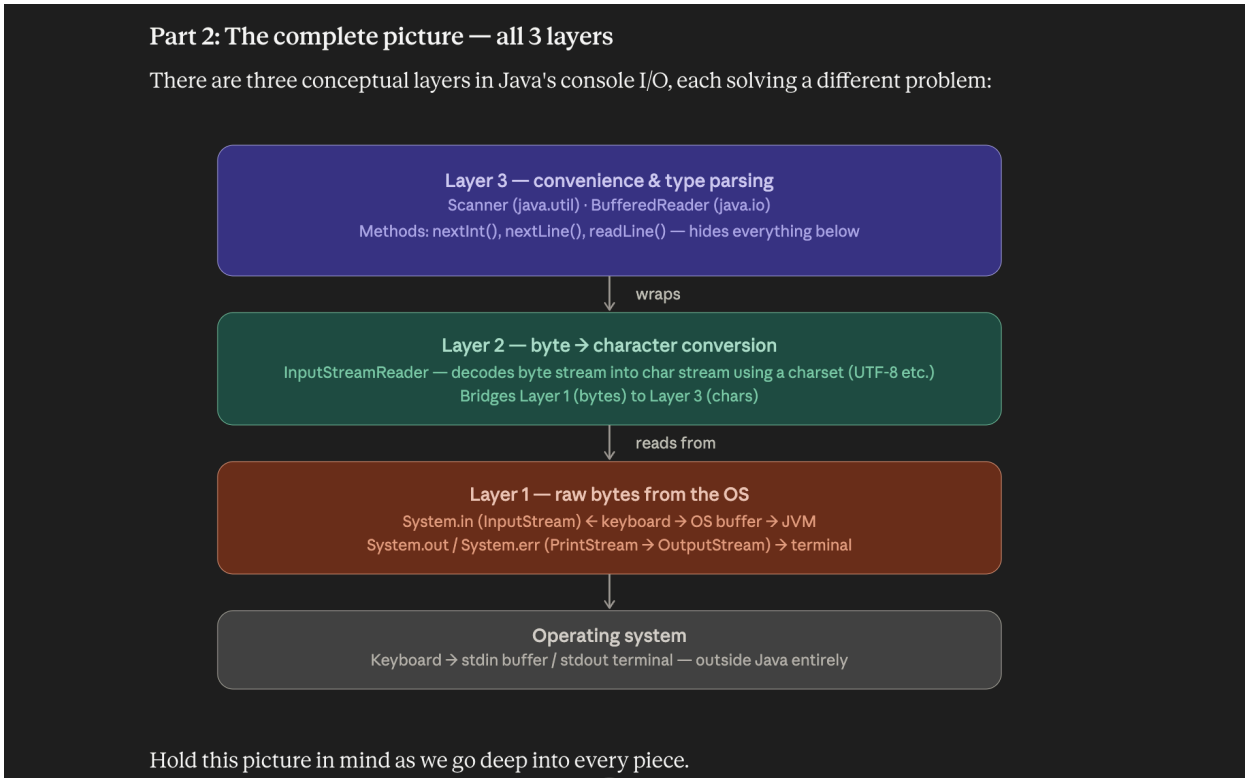
**Fix:** Add a dummy `sc.nextLine()` to consume the newline after `nextInt()`:

```
java
int age = sc.nextInt();
sc.nextLine(); // consume the leftover newline
String name = sc.nextLine(); // now reads correctly
```

## Part 1: The Philosophy — why Java's I/O is complex

Before anything: Java was designed as a *server-side language*, not a terminal scripting tool. Its input wasn't meant to come from a keyboard — it was meant to come from network sockets, files, databases, and API payloads. So Java's I/O primitives were built at a low level, and convenience wrappers came later.

This means when you understand Java's I/O, you're actually understanding how *all* I/O works at the OS level. That knowledge transfers to networking, file handling, Spring's request/response model — everything.




## Part 3: Dissecting `System.out.println()` — what actually happens

This is the most-typed line in Java history and almost nobody knows what it actually is.

The breakdown: `System` → `out` → `println("hello")`

**`System` is a class.** It lives in `java.lang`, which is auto-imported by every Java file — you never write `import java.lang.System`. The JVM compiler injects it silently.

**`out` is a static field inside that class.** Since it's `static`, you access it via the class name directly. Its declaration in Java's source looks like:

java
public static final PrintStream out = null;

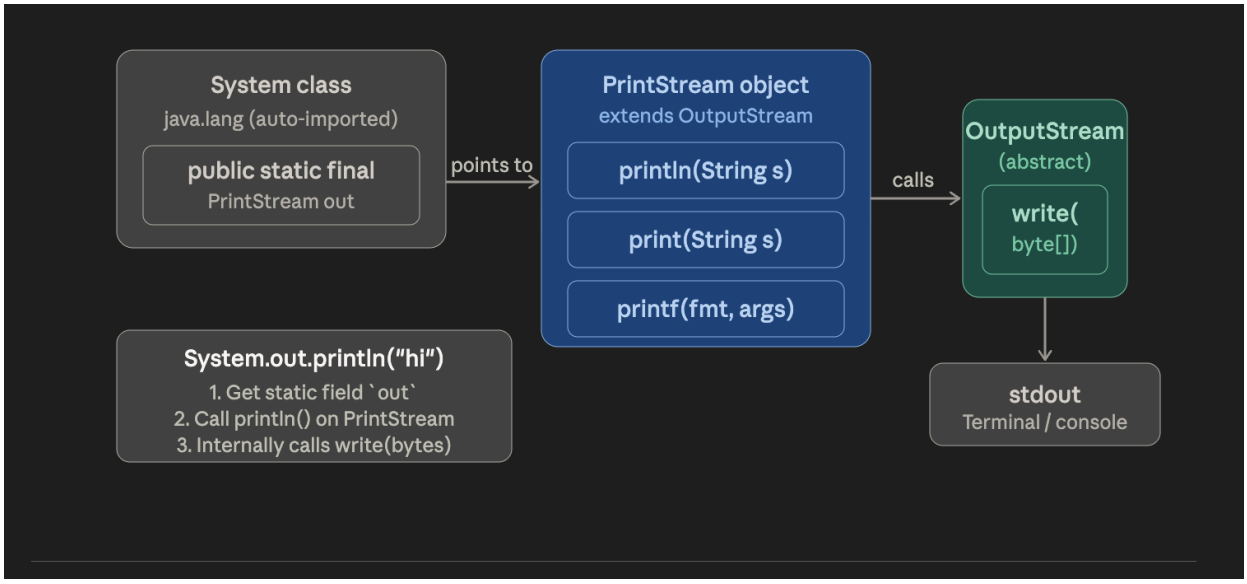
It's `public` (accessible anywhere), `static` (belongs to the class, not an instance), and `final` (can't be reassigned). The `null` you see in source is just the declaration — the JVM wires it up to the actual terminal output stream at startup, before your `main()` ever runs.

**`PrintStream` is the actual class.** `out` is a *reference variable* of type `PrintStream`. So `System.out` gives you a `PrintStream` object. That object has methods on it — and one of them is `println()`.

**`println()` vs `print()` vs `printf()`** — all three live on `PrintStream`:

- `println(x)` — prints x, then appends a newline (`\n`). Cursor moves to next line.
- `print(x)` — prints x, stays on the same line.
- `printf("Name: %s, Age: %d", name, age)` — formatted output, like C's printf. Used when you need specific decimal precision, padding, etc.

Internally, all three eventually call a `write()` method — because `PrintStream` extends `OutputStream`, and `OutputStream`'s contract is to write bytes. So `println("hello")` converts the string to bytes using the platform's default charset, then writes those bytes to stdout.




## Part 4: `System.err` — what it is and why it exists

`System.err` is also a `PrintStream`, declared identically to `out`:

java
public static final PrintStream err = null;

Both `out` and `err` write to your terminal by default. They look the same. But they are different OS-level **file descriptors**:

- `System.out` → **stdout** (file descriptor 1)
- `System.err` → **stderr** (file descriptor 2)

Why does this matter? At the OS level, you can redirect them independently:

bash
java MyApp > output.txt        # stdout goes to file, stderr still shows on terminal
java MyApp 2> errors.txt       # stderr goes to file, stdout still on terminal
java MyApp > out.txt 2> err.txt  # both redirected separately

This is why Spring Boot and every serious Java app uses a logging library (SLF4J, Logback, Log4j) — those libraries write errors to stderr and info to stdout, so ops teams can route them differently. When you see red-coloured output in your IDE's console, that's your IDE distinguishing stderr from stdout visually.

java
// These both print to terminal when running locally
System.out.println("User logged in");        // stdout — normal flow
System.err.println("DB connection failed");  // stderr — error flow

**Best practice**: use `System.err` for errors, exceptions, and unexpected states. Use `System.out` for normal program output. But in production Java (Spring Boot, etc.), use a proper logger instead of either.

## Part 5: What is a Stream?

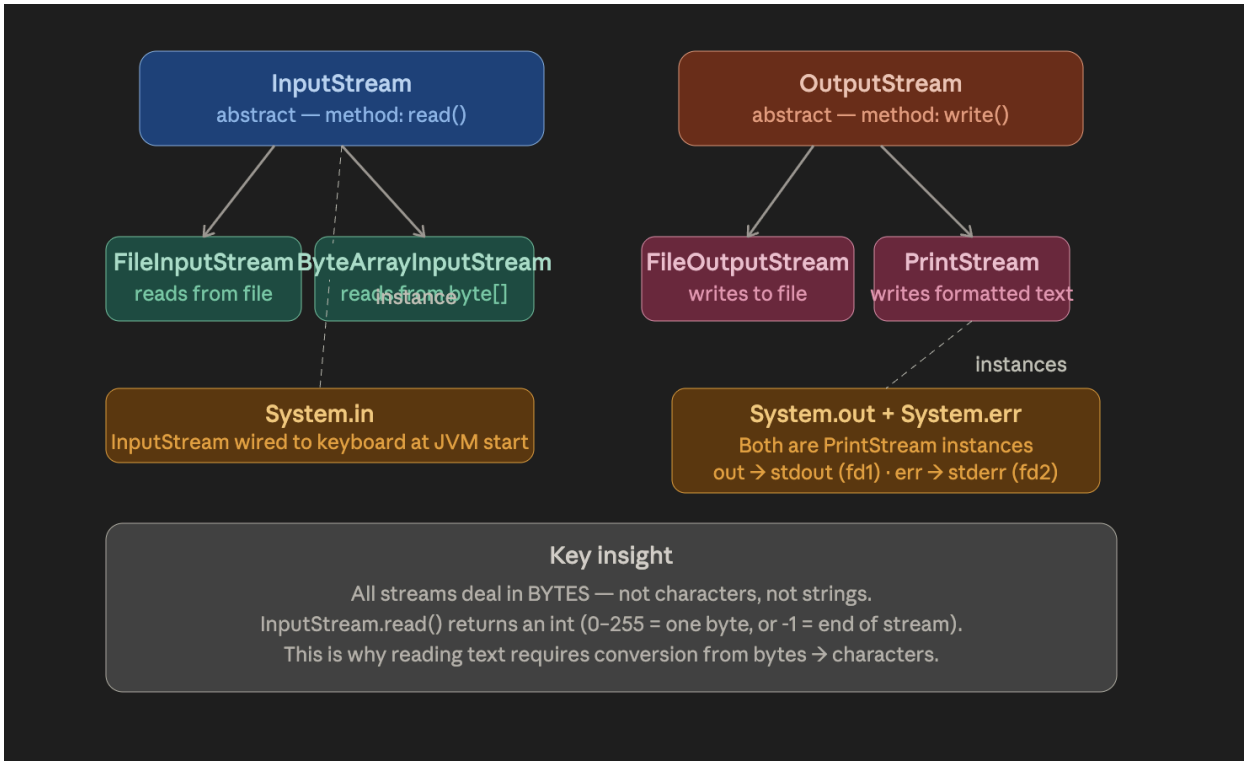
Before we touch `System.in`, we need to deeply understand *streams* — because everything in Java I/O is built on this concept.

A stream is a **sequence of data flowing in one direction** through a channel. Think of it like a Unix pipe: data comes out one end continuously. You don't get "the whole thing at once" — you read piece by piece as it arrives.

Two directions:

- **InputStream** — data flows *into* your program (reading)
- **OutputStream** — data flows *out of* your program (writing)

Both `InputStream` and `OutputStream` are **abstract classes**. They define the *contract* (what operations you can do) but not the *source/destination* (where data comes from or goes to). The concrete implementations specify that:




## Part 6: `System.in.read()` — the raw, painful approach

`System.in` is an `InputStream`. `InputStream` has a method `read()`. Let's understand what it actually does.

java
int c = System.in.read();  // returns next byte as int, or -1 at end-of-stream

When you type `A` on the keyboard:

1. OS captures the keypress and puts it in the stdin buffer
2. The OS converts it to the ASCII value: `A` = 65
3. `System.in.read()` reads that one byte: returns `65`
4. If you want the character `'A'`, you cast: `(char) 65` = `'A'`

The `int` return type (not `byte`) is intentional — it needs to be able to return `-1` to signal "no more data" (end of stream), and a `byte` can only hold -128 to 127.

**The loop problem**: If you typed `Aditya`, the OS buffer holds:

```
65 100 105 116 121 97 10
 A   d   i   t   y  a  \n
```

`read()` only returns *one byte at a time*. So to read the whole word, you'd loop:

java
```
import java.io.IOException;

public class Demo {
    public static void main(String[] args) throws IOException {
        StringBuilder sb = new StringBuilder();
        int c;
        while ((c = System.in.read()) != '\n') {  // read until Enter key
            sb.append((char) c);
        }
        System.out.println(sb.toString());
    }
}
```

This works, but every single `read()` call hits the OS. For 1000 characters, that's 1000 OS syscalls. That's slow.

Also: you can't read an `int` directly. If you type `25`, you get back two bytes: `50` (`'2'`) and `53` (`'5'`). You'd have to manually parse that string "25" into the integer 25. That's the core limitation.

---

## Part 7: The Reader family — enter characters

Java introduced a second parallel hierarchy alongside streams: the **Reader/Writer** hierarchy. The key difference:

### Part 7: The Reader family — enter characters

Java introduced a second parallel hierarchy alongside streams: the **Reader/Writer** hierarchy. The key difference:

| | Stream (InputStream/OutputStream) | Reader/Writer |
|---|---|---|
| Unit | Byte (8 bits) | Character (16 bits, Unicode) |
| Type returned by read() | int (0–255) | int (0–65535) |
| Best for | Binary data, images, network | Text, keyboard input |

`Reader` is also abstract, with concrete subclasses for each source:

```
Reader (abstract)
├── InputStreamReader   ← converts byte stream → char stream
├── BufferedReader      ← adds buffering + readLine()
├── FileReader          ← reads characters from a file
└── StringReader        ← reads from a String
```

The critical one for keyboard input is `InputStreamReader`, because it bridges Layer 1 (bytes) to Layer 2 (characters).

## Part 8: `InputStreamReader` — the byte-to-char bridge

`InputStreamReader` wraps an `InputStream` and converts its bytes to characters using a charset decoder. You construct it like this:

java
InputStreamReader isr = new InputStreamReader(System.in);
// or specify charset explicitly:
InputStreamReader isr = new InputStreamReader(System.in, StandardCharsets.UTF_8);

Internally, it:

1. Calls `System.in.read()` to get bytes
2. Applies the charset (UTF-8 by default) to decode those bytes into Unicode characters
3. Returns characters instead of bytes

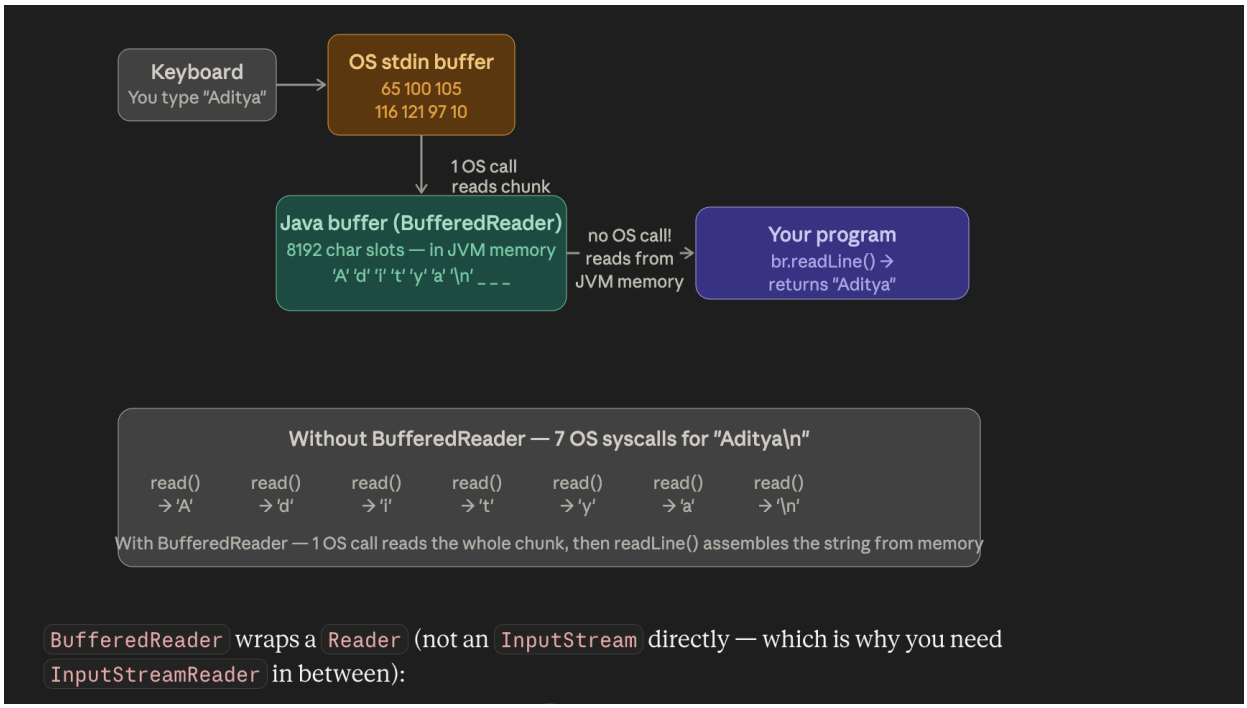
For ASCII characters like `A`, one byte = one character (65 → `'A'`). For multibyte characters like `é` (UTF-8 encodes this as 2 bytes: `0xC3 0xA9`), `InputStreamReader` handles the multi-byte decoding automatically and gives you back one `char`.

This is a *decorator* pattern — `InputStreamReader` doesn't replace `System.in`, it *wraps* it. The actual data still comes from the keyboard via `System.in`; `InputStreamReader` just reinterprets the byte stream as characters.

## Part 9: `BufferedReader` — the performance layer

Even with `InputStreamReader`, we still read one character at a time. That means one OS syscall per character.

`BufferedReader` solves this by adding an internal memory buffer (default: 8192 characters). Here's what happens:




`BufferedReader` wraps a `Reader` (not an `InputStream` directly — which is why you need `InputStreamReader` in between):

```
// Full chain: keyboard → bytes → chars → buffered chars → String
BufferedReader br = new BufferedReader(new InputStreamReader(System.in));

String line = br.readLine();  // reads until \n, returns the String without \n
```

`readLine()` returns `null` when the stream ends (Ctrl+D on Unix, Ctrl+Z on Windows). Always handle that:

```
String line = br.readLine();
if (line != null) {
    System.out.println(line);
}
```

## Part 10: `BufferedReader` limitations

Two real pain points:

**1. Only reads Strings.** `readLine()` always returns a `String`. If you want an `int`, you must parse manually:

```java
String s = br.readLine();     // "42"
int n = Integer.parseInt(s);  // 42  (throws NumberFormatException if not a number)

// or for double:
double d = Double.parseDouble(br.readLine());
```

**2. Verbose setup.** Three objects just to read from keyboard:

```java
InputStreamReader isr = new InputStreamReader(System.in);
BufferedReader br = new BufferedReader(isr);
// or one-liner:
BufferedReader br = new BufferedReader(new InputStreamReader(System.in));
```

This verbosity pushed Java to create a higher-level tool.

## Part 11: `Scanner` — the convenient wrapper

Java 1.5 added `Scanner` in `java.util` (not `java.io` — it's a utility, not a stream primitive). It wraps the entire chain and adds tokenisation and type parsing.

```java
import java.util.Scanner;

Scanner sc = new Scanner(System.in);  // takes an InputStream in its constructor
```

Internally, `Scanner` wraps an `InputStreamReader` and uses a buffer, but it also:

- Tokenises the input using a delimiter (whitespace by default)
- Parses tokens into types automatically

**All the methods:**

```java
String line  = sc.nextLine();    // reads whole line until \n (like readLine())
String word  = sc.next();        // reads next whitespace-delimited token
int n        = sc.nextInt();     // reads next token, parses as int
double d     = sc.nextDouble();  // parses as double
boolean b    = sc.nextBoolean(); // parses "true"/"false"
long l       = sc.nextLong();    // etc.
```

`next()` vs `nextLine()` — the subtle bug everyone hits:

```java
int age = sc.nextInt();      // reads "25", but the \n after it stays in the buffer!
String name = sc.nextLine(); // reads that leftover \n → returns "" immediately!

// Fix: consume the leftover newline
int age = sc.nextInt();
sc.nextLine();               // discard the \n
String name = sc.nextLine(); // now reads the actual next line
```

This happens because `nextInt()` reads `25` but *stops before* the newline. The newline sits in the buffer. The next `nextLine()` reads up to that newline and returns an empty string.

`Scanner` **can read from multiple sources** — its constructor accepts `InputStream`, `File`, `String`, `Path`:

```java
Scanner fromFile    = new Scanner(new File("data.txt"));
Scanner fromString  = new Scanner("10 20 30");
Scanner fromKeyboard = new Scanner(System.in);
```

## Part 12: Scanner vs BufferedReader — when to use which

This is a real engineering decision, not just academic:

| Dimension | BufferedReader | Scanner |
|---|---|---|
| **Performance**<br>speed critical? | **Faster**<br>minimal overhead, no regex | **Slower**<br>tokenises + regex under hood |
| **Type parsing**<br>read int/double? | **Manual**<br>Integer.parseInt(readLine()) | **Built-in**<br>nextInt(), nextDouble() etc. |
| **Thread safety**<br>multiple threads? | **Synchronized**<br>read() is thread-safe | **Not synchronized**<br>not safe across threads |
| **Setup verbosity**<br>boilerplate? | **More verbose**<br>new BR(new ISR(System.in)) | **Concise**<br>new Scanner(System.in) |
| **Use when** | **BufferedReader**<br>Competitive programming<br>Large input files<br>Raw text line-by-line | **Scanner**<br>General user input<br>Reading mixed types<br>Quick scripts / learning |

The reason `Scanner` is slower: it uses Java's `regex` engine internally to match tokens and parse types. Every `nextInt()` call runs a regex pattern match. `BufferedReader.readLine()` just scans for `\n` and returns the slice. For 10M lines of input, this difference is measurable.

In competitive programming (Codeforces, LeetCode with Java), you'll often see people use `BufferedReader` + `Integer.parseInt` specifically to avoid `Scanner`'s overhead.

## Part 13: The full picture — all three approaches in code

Here's every approach side by side so you can see the evolution:

```
// ──────────────────────────────────────────────────────────────
// Approach 1: System.in.read() — raw bytes
// Painful, educational only
import java.io.IOException;

public class Approach1 {
    public static void main(String[] args) throws IOException {
        System.out.print("Enter a character: ");
        int b = System.in.read();      // one byte as int
        System.out.println("Byte value: " + b);            // e.g. 65
        System.out.println("Character: " + (char) b);      // e.g. A
        // Can't easily read "Aditya" — need a loop
    }
}

// ──────────────────────────────────────────────────────────────
// Approach 2: BufferedReader — fast, strings only
import java.io.*;

public class Approach2 {
    public static void main(String[] args) throws IOException {
        BufferedReader br = new BufferedReader(new InputStreamReader(System.in));

        System.out.print("Name: ");
        String name = br.readLine();          // "Aditya"

        System.out.print("Age: ");
        int age = Integer.parseInt(br.readLine());  // "25" → 25

        System.out.println(name + " is " + age);
```

```
    }
}

// ——————————————————————————————————————————————————————————
// Approach 3: Scanner — convenient, slightly slower
import java.util.Scanner;

public class Approach3 {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);  // no throws needed

        System.out.print("Name: ");
        String name = sc.nextLine();          // "Aditya"

        System.out.print("Age: ");
        int age = sc.nextInt();               // 25 — directly, no parseInt
        sc.nextLine();                        // consume leftover \n

        System.out.print("Score: ");
        double score = sc.nextDouble();       // 98.5

        System.out.println(name + ", " + age + ", " + score);
        sc.close();  // always close Scanner when done
    }
}
```

## Part 14: One more thing — always close `Scanner`

`Scanner` implements `Closeable`. When you're done, call `sc.close()`. Better: use try-with-resources:

### Part 14: One more thing — always close `Scanner`

`Scanner` implements `Closeable`. When you're done, call `sc.close()`. Better: use try-with-resources:

```java
try (Scanner sc = new Scanner(System.in)) {
    String name = sc.nextLine();
    System.out.println("Hello, " + name);
}  // sc.close() called automatically here
```

`BufferedReader` too:

```java
try (BufferedReader br = new BufferedReader(new InputStreamReader(System.in))) {
    String line = br.readLine();
    System.out.println(line);
}
```

## Summary — the mental model

Every piece connects:

1. **Keyboard → OS buffer** — keystrokes converted to bytes, stored in OS's stdin buffer
2. **`System.in`** — Java's static `InputStream` that reads from that OS buffer, one byte at a time
3. **`InputStreamReader`** — wraps `System.in`, converts byte stream → char stream using charset
4. **`BufferedReader`** — wraps `InputStreamReader`, chunks the reads (1 OS call for 8192 chars), adds `readLine()`
5. **`Scanner`** — wraps the whole chain, tokenises input, parses types — convenience at the cost of regex overhead
6. **`System.out`** / **`System.err`** — static `PrintStream` objects writing to stdout (fd1) and stderr (fd2)

When you write `new Scanner(System.in)`, all five layers are in play simultaneously. Scanner just hides layers 2–4 inside its constructor.